Comprehensive Benchmark Metrics for Weighted A^* 15-Puzzle Solver

Md. Fuad Al Alam
Student ID: 2205167

July 7, 2026

1 Overview and Methodology

This document outlines performance metrics across 10 separate initial board state configurations for a 15-Puzzle. The Linear Conflict heuristic is tracked across varying heuristic weights $w \in \{1.0, 1.2, 1.5, 2.0\}$. All baseline raw figures represent the nodes explored and the minimum number of moves to reach the goal state.

2 Empirical Trend Analysis

An examination of the benchmark data across all 10 configurations reveals critical algorithmic trends governing Weighted A^* search dynamics on the 15-puzzle:

2.1 Search Complexity vs. Heuristic Weight

- **Exponential Reduction in Frontier Size:** As the heuristic weight w scales up from 1.0 to 2.0, the volume of nodes explored experiences an exponential drop for harder configurations. Increasing the weight forces the algorithm to act more greedily, heavily prioritizing depth-first progress towards the goal state. This massive contraction in state evaluation serves as the primary mechanism for computational efficiency.

2.2 Path Optimality vs. Heuristic Weight

- **Degradation of Solution Quality:** In compliance with the theoretical sub-optimality bound ($\text{Cost} \leq w \cdot \text{Cost}^*$), higher values of w correspond directly to inflated move counts. While $w = 1.0$ guarantees mathematically optimal trajectories, moving to $w = 2.0$ systematically balloons step counts (e.g., scaling Input 7's length from an optimal 53 moves to an inefficient 67 moves).
- **The Core Bounded Sub-optimal Trade-off:** The data establishes a clear trade-off profile: runtime memory safety and speed are directly bought at the expense of step-efficiency.

15-Puzzle Configurations

Goal Configuration

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	0

Initial Input States

Input 1
3 4 0 12
2 1 8 15
9 5 6 7
13 10 14 11

Input 2
1 2 7 3
5 6 4 15
14 0 10 12
9 13 8 11

Input 3
6 10 7 3
1 2 4 8
9 15 11 12
13 5 0 14

Input 4
5 1 2 4
9 10 3 6
13 14 7 8
12 11 15 0

Input 5
12 1 2 4
14 0 3 7
9 10 8 6
13 5 11 15

Input 6
2 9 4 15
6 0 13 7
10 1 12 11
14 5 3 8

Input 7
10 14 9 4
8 11 15 7
1 5 12 3
6 2 0 13

Input 8
15 5 3 0
10 1 12 2
6 14 8 4
9 13 11 7

Input 9
1 12 0 2
6 7 13 10
4 11 9 15
3 14 8 5

Input 10
1 14 5 6
12 11 0 8
13 10 9 4
2 15 3 7

3 Per-Input Empirical Results

Input Configuration 1

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	28	28	28	28
Solution Move Count	24	24	24	24

Input Configuration 2

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	105	108	113	131
Solution Move Count	21	21	21	23

Input Configuration 3

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	3,037	2,632	1,292	636
Solution Move Count	31	31	31	35

Input Configuration 4

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	109	104	84	129
Solution Move Count	24	24	24	24

Input Configuration 5

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	5,106	2,700	526	852
Solution Move Count	34	34	34	34

Input Configuration 6

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	175,217	97,301	7,624	1,166
Solution Move Count	46	46	48	52

Input Configuration 7

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	896,223	169,327	13,774	5,629
Solution Move Count	53	53	55	67

Input Configuration 8

Metrics	w = 1.0	w = 1.2	w = 1.5	w = 2.0
Nodes Explored	22,430	6,920	398	134
Solution Move Count	41	41	41	41

Input Configuration 9

Metrics	**w = 1.0**	**w = 1.2**	**w = 1.5**	**w = 2.0**
Nodes Explored	186,135	42,732	883	406
Solution Move Count	52	52	56	60

Input Configuration 10

Metrics	**w = 1.0**	**w = 1.2**	**w = 1.5**	**w = 2.0**
Nodes Explored	962,180	465,911	46,360	2,545
Solution Move Count	51	51	55	61

Observations

- **Optimality ($W = 1.0$):** Setting $W = 1.0$ corresponds to the standard A^* search. It always guarantees the optimal solution (lowest cost). However, for harder configurations (e.g., Inputs 6, 7, 9, 10), it significantly inflates the number of explored nodes, reaching up to almost a million nodes in the worst case tested (Input 10).
- **Efficiency ($W > 1.0$):** As the weight increases (1.2, 1.5, 2.0), the algorithm heavily prioritizes the heuristic, causing it to act more like a greedy best-first search. This generally reduces the total nodes explored by a significant magnitude. For instance, in Input 10, the explored nodes fell from $962,180$ ($W = 1.0$) down to just $2,545$ ($W = 2.0$).
- **Sub-optimality Trade-off:** The rapid node reduction comes at the expense of solution cost. In almost all complex inputs, increasing W above 1.0 begins to yield suboptimal paths. For example, in Input 7, the optimal cost of 53 ($W = 1.0, 1.2$) jumps to 67 ($W = 2.0$).